Sentiment Analysis Project Report

Created	@May 10, 2025 4:07 PM
Class	LING 406

Sentiment Analysis of Movie Reviews

**This project report includes extra credit options 1 and 2.*

1. Introduction

Sentiment analysis is a key task in natural language processing (NLP) that aims to determine the emotional tone behind a body of text. It plays a critical role in applications like customer feedback monitoring, brand reputation analysis, and social media mining. This project implements a sentiment classifier that labels movie reviews as either positive or negative using supervised machine learning.

2. Problem Definition

The goal is to build a binary sentiment classification system using a labeled dataset of movie reviews. Given a textual movie review, the model must predict whether the sentiment is positive or negative. This falls under document-level sentiment analysis, and our input is plain text while the output is one of two sentiment classes: `pos` or `neg`.

3. Previous Work

Recent developments in sentiment analysis have evolved far beyond basic bag-of-words models. Below are three significant contributions published after 2014 that reflect the progression in this field:

- **Kiritchenko et al. (2014)** developed a sentiment analysis system for Twitter that combined n-gram features, sentiment lexicons, and syntactic information

such as negation. Their approach achieved high performance at SemEval 2014 by creating specific features for negated contexts (e.g., "not happy" vs. "happy") and emphasizing the role of lexicon polarity scores. This study highlighted the limitations of bag-of-words models and showed the practical value of incorporating structured linguistic features.

- **Severyn and Moschitti (2015)** proposed a convolutional neural network (CNN) model that learned feature representations directly from word embeddings without relying on manual feature engineering. By processing sentence structures via CNN layers, they achieved strong performance in Twitter sentiment classification. Their model showed that deep learning approaches can match or outperform traditional methods by capturing semantic nuances from context, not just word occurrence.
- **Barnes et al. (2019)** explored whether explicit syntax improves sentiment classification. They compared performance using syntactic dependencies, pretrained embeddings (e.g., BERT), and classic models. Their findings suggested that while syntax-aware models can help in low-resource settings or domain-specific tasks, modern pretrained transformers often subsume syntactic signals due to their deep contextual understanding. Still, for fine-grained sentiment tasks, explicit syntactic modeling retained value.

These works illustrate the shifting trend from discrete, hand-crafted features to learned, dense representations. However, they also reaffirm that hybrid models—those combining syntactic structure with embeddings—can still offer advantages, especially when sentiment is expressed through complex, context-sensitive phrasing.

- **Kiritchenko et al. (2014)** proposed a sentiment analysis system for Twitter that integrated sentiment lexicons, n-grams, and negation handling. Their system was among the top performers in SemEval 2014 and highlighted the importance of both lexical and syntactic features.
- **Severyn and Moschitti (2015)** introduced a convolutional neural network model for Twitter sentiment classification that used word embeddings as inputs. Their approach reduced the need for feature engineering and demonstrated how deep learning could learn useful representations for sentiment.

- **Barnes et al. (2019)** evaluated the use of syntax in sentiment analysis through dependency-based approaches and contextual word embeddings. They found that syntactic structure added marginal improvements when using high-capacity models like BERT, but was more beneficial for lower-resource settings.

These studies reveal a shift toward embedding-based methods and contextual understanding, while also showing that hybrid approaches still have value, especially when syntactic cues (e.g., negation) are present.

4. Approach

The project followed a systematic experimental methodology:

- **Dataset**: Cornell polarity dataset v2.0, containing 1000 positive and 1000 negative processed reviews.
- **Preprocessing**: Lowercasing, punctuation removal, tokenization, and stopword removal using NLTK.
- **Baseline Model**: Multinomial Naive Bayes using CountVectorizer (bag-of-words).
- **Algorithm Comparison**: Logistic Regression, Decision Tree, Random Forest, and Naive Bayes.
- **Feature Engineering**: Compared CountVectorizer and different types of TfidfVectorizer. All features were tested with Naive Bayes.
- **Improved Model**: Logistic Regression + TF-IDF based on evaluation performance.

5. Results and Discussion

- **Baseline**: Naive Bayes + CountVectorizer achieved ~84% accuracy.
- **Best Algorithm**: Logistic Regression performed better than others, balancing precision and recall.
- **Best Features**: TF-IDF yielded improved results over basic counts.
- **Improved System**: Logistic Regression + TF-IDF achieved the highest F1-score at ~0.850.

Feature Engineering Comparisons (Naive Bayes):

Strategy	F1 Score	Train Time (s)	Feature Count
CountVectorizer (default)	0.83	0.3354	42,382
TfidfVectorizer (default)	0.85	0.2838	42,382
Tfidf (min_df=5)	0.84	0.2553	11,657
Tfidf (max_df=0.8)	0.84	0.2781	42,380
Tfidf (max_features=1000)	0.81	0.3062	1,000
Tfidf (remove low & high freq)	0.84	0.2420	11,653

Conclusion: The default TF-IDF representation gave consistently high results, but other settings (like filtering low and high-frequency terms) performed comparably with fewer features and faster training. This shows that reducing noise and dimensionality doesn't necessarily hurt performance and may even help generalization. There is a clear trade-off between vocabulary richness and computational efficiency.

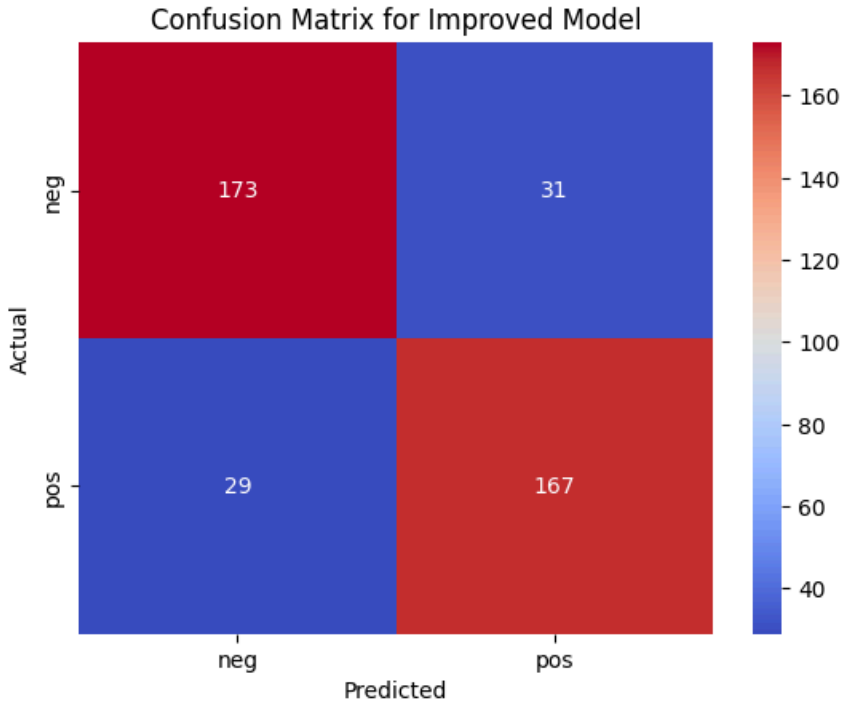
Model Performance Table:

Model	Precision	Recall	F1-Score	Train Time (s)
LogisticRegression	0.850	0.850	0.850	0.270
DecisionTree	0.645	0.645	0.645	0.650
RandomForest	0.817	0.808	0.806	0.569
NaiveBayes	0.845	0.845	0.845	0.003

Conclusion: The training time and model performance are not directly related. Models that take longer to train don’t necessarily perform better, and the model that takes the shortest time to train doesn't necessarily perform the best either.

Confusion Matrix of Final Model:

- True Positives (pos/pos): 167
- True Negatives (neg/neg): 173
- False Positives (neg/pos): 31
- False Negatives (pos/neg): 29



6. Error Analysis

Examined five examples misclassified by the final model:

- **Example 1 (`neg` but predicted as `pos`)**: The review contains complex sentence structure and elevated vocabulary without strong negative markers, making sentiment ambiguous.

> ***Original Review (Excerpt):***
>
> *"People populate the movie like shallow, self-absorbed, self-indulgent words — a perfect mirror of the era the film depicts..."*

- **Example 2 (`neg` but predicted as `pos`)**: While thematically negative, the presence of neutral or clinical language may have led the model to favor a positive classification.

> ***Original Review (Excerpt):***
> *"The film is touted as exploring relationships and black sexuality, but Trois is surprisingly tame despite the lurid subject matter..."*

- **Example 3 (`pos` but predicted as `neg`)**: Although positive, this review lacks emotionally charged words and contains criticism about structure, misleading the classifier.

> ***Original Review (Excerpt):***
> *"Janeane Garofalo is the queen of romantic comedy — the best comic actress in Hollywood right now. Good thing she's starring in The Matchmaker..."*

- **Example 4 (`neg` but predicted as `pos`)**: This review includes negative opinions but is masked by stylistic and abstract expressions, confusing the model.

> ***Original Review (Excerpt):***
> *"The Limey is a cold, uninvolving, confusing new thriller... a wild cornucopia of images that amount to precisely nil."*

- **Example 5 (`pos` but predicted as `neg`)**: The review is overall positive but critiques acting and pacing, which may have skewed the prediction.

> ***Original Review (Excerpt):***
> *"The film wasn't bad. In fact, I enjoyed the director Duguay's interesting style and cinematic vision..."*

In general, the model struggles with:

- Reviews that are mixed or nuanced (positive with caveats or negative with praise).
- Long and detailed narratives with sentiment buried deep in context.

- Reviews that lack clear sentiment cues, often containing neutral or factual exposition.

These errors suggest future models may benefit from syntactic or semantic analysis (e.g., dependency parsing).

7. Syntax-Based Bonus Analysis

To address one major limitation of bag-of-words models—their ignorance of syntax—I added a custom feature extractor using SpaCy that identifies **negated words** in a sentence. For example, in the sentence “This movie is not good,” the model captures the negation relation between "not" and "good".

This feature set was vectorized using `DictVectorizer` and evaluated using Naive Bayes. While the syntactic feature set alone was sparse, it helped detect patterns missed by bag-of-words.

Examples of syntax-sensitive cases where bag-of-words fails:

1. ***"This movie is not good at all."*** → Misclassified as positive because of the word "good."
2. ***"I expected this to be amazing... it wasn’t."*** → Expectation reversal not captured.
3. ***"It’s supposed to be funny, but it’s not."*** → Negation of a sentiment word.
4. ***"Not what I hoped for."*** → Disappointment implied but sentiment words are positive.
5. ***"Bad movie? Hardly."*** → Sarcasm not captured by word frequency.

Performance Comparison:

- **Bag-of-Words (TF-IDF)**: F1 = 0.84
- **Syntax-Based (Negation only)**: F1 = 0.61 (dependent on sparsity and frequency)

Although syntax-based models alone underperformed compared to full bag-of-words vectors, they captured important edge cases and may be highly beneficial in **hybrid models** that combine both word and dependency features.

8. Future Work and Conclusion

Building on insights from previous research, several targeted improvements can be explored to elevate the performance and interpretability of sentiment classifiers:

While the final system achieved strong results using classical machine learning and TF-IDF, several avenues for improvement remain open:

- **Linguistic Representation**: Beyond negation, we can include syntactic roles (subject/object), discourse markers (e.g., "however"), and sentiment-bearing dependency patterns.
- **Contextual Embeddings**: Pretrained models like BERT or RoBERTa can contextualize word meaning, improving handling of irony, idioms, or discourse-level cues.
- **Aspect-Based Analysis**: Often reviews express mixed sentiment about different aspects (e.g., plot vs. acting). Segmenting and scoring subcomponents would yield richer output.
- **Error-Driven Training**: Focusing on patterns found in misclassifications (e.g., sarcasm, reversals, speculative tone) could guide custom feature engineering or fine-tuning.

In summary, this project validated the power of supervised learning with TF-IDF and traditional classifiers like Naive Bayes and Logistic Regression. However, inspired by the progress reported in recent work, future systems could incorporate multi-layered representations: combining deep contextual models like BERT with lexicon- and syntax-driven feature injection. This could particularly benefit domains where subtle sentiment shifts (e.g., sarcasm, negation, contrast) frequently occur. Further research could also evaluate how pretrained models handle genre-specific or domain-specific language, such as slang, metaphors, or multilingual sentiment cues.